

Relatório de Auditoria de Segurança — Protocolo ARCA

Ferramenta: Slither v0.10.x (análise estática de contratos Solidity)

Data: 22/04/2026

Autor: Luciano Rodrigues de Moraes

Curso: Residência em TIC 29 – Web 3.0

Contratos analisados: ARCAToken.sol, ARCABadge.sol, ARCAStaking.sol, ARCAGovernance.sol

Resultado: 28 contratos analisados com 101 detectores — 68 resultados encontrados (maioria em dependências OpenZeppelin)

Sumário Executivo

A análise estática com Slither identificou **nenhuma vulnerabilidade crítica** nos contratos do Protocolo ARCA. Os alertas encontrados se dividem em:

Severidade	Quantidade	Origem
Alta	0	—
Média	3	Contratos ARCA (padrão esperado)
Baixa	4	Contratos ARCA
Informativo	61	Dependências OpenZeppelin/Chainlink

Achados nos Contratos ARCA

[MÉDIA] Reentrancy em ARCABadge.mintBadge — reentrancy-no-eth

Arquivo: contracts/ARCABadge.sol , linha 51-64

Descrição: A função `mintBadge` chama `_safeMint` (que pode invocar `onERC721Received` em contratos externos) antes de atualizar o mapping `possuiBadge`. Isso cria uma janela teórica de reentrância.

```
_safeMint(osc, novoId);           // chamada externa
possuiBadge[osc] = true;          // estado atualizado DEPOIS
```

Risco real: Baixo — a função é `onlyOwner`, portanto apenas o administrador pode chamá-la. Não há ETH envolvido.

Mitigação aplicada: Controle de acesso `onlyOwner` limita o vetor de ataque.

Recomendação: Mover `possuiBadge[osc] = true` para antes de `_safeMint` (padrão checks-effects-interactions).

[MÉDIA] Igualdade estrita em ARCAStaking — incorrect-equality

Arquivo: contracts/ARCAStaking.sol , linha 174

Descrição: Uso de `==` para comparar `info.quantidade == 0` em vez de `<= 0`.

```
if (info.quantidade == 0) return 0;
```

Risco real: Mínimo — `uint256` não pode ser negativo, então `== 0` é equivalente a `<= 0` neste contexto.

Mitigação: Sem impacto funcional; é uma questão de estilo/boas práticas.

[MÉDIA] Uso de `block.timestamp` — timestamp

Arquivos: ARCAGovernance.sol (linhas 109, 135), ARCAStaking.sol (linha 99)

Descrição: O Slither alerta para o uso de `block.timestamp` em comparações de prazo, pois mineradores/validadores podem manipular o timestamp em até ~15 segundos.

```
require(block.timestamp <= p.dataFim, "Periodo de votacao encerrado.");
require(block.timestamp >= info.timestamp + LOCK_PERIOD, "Lock ativo.");
```

Risco real: Baixo — em Proof of Stake (Ethereum pós-Merge), a manipulação de timestamp é muito mais difícil. Para períodos de lock de dias/semanas, uma variação de 15 segundos é irrelevante.

Mitigação: Aceitável para o contexto de MVP educacional. Em produção, considerar uso de número de bloco.

[BAIXA] Valor de retorno ignorado em ARCAStaking — `unused-return`

Arquivo: `contracts/ARCAStaking.sol`, linha 128

Descrição: A função `latestRoundData()` do Chainlink retorna 5 valores; apenas o preço (`answer`) é utilizado.

```
(, int256 preco,,, ) = priceFeed.latestRoundData();
```

Risco real: Baixo — os campos ignorados (`roundId`, `startedAt`, `updatedAt`, `answeredInRound`) são usados para validação de staleness em produção. Para testnet educacional, é aceitável.

Recomendação: Em produção, validar `updatedAt` para garantir que o preço não está desatualizado.

[BAIXA] Literais com muitos dígitos — `too-many-digits`

Arquivo: `contracts/ARCAStaking.sol`, linhas 145, 147

Descrição: Constantes numéricas grandes sem separadores de legibilidade.

```
preco > 3000000000000 // $3.000 em 8 decimais Chainlink
preco > 2000000000000 // $2.000 em 8 decimais Chainlink
```

Risco real: Nenhum — apenas questão de legibilidade.

Recomendação: Usar `300_000_000_000` (underscore como separador, suportado em Solidity ^{0.8.x}).

[INFORMATIVO] Versão do compilador — `pragma / solc-version`

Descrição: Todos os contratos usam `pragma solidity ^0.8.24`, que é uma versão recente e segura. O Slither alerta que o `pragma` permite versões futuras (^), mas isso é padrão na indústria.

Proteções Implementadas

Proteção	Contrato	Implementação
Anti-reentrância	ARCAStaking	<code>ReentrancyGuard</code> (OpenZeppelin)
Anti-reentrância	ARCAGovernance	<code>ReentrancyGuard</code> (OpenZeppelin)
Controle de acesso	Todos	<code>Ownable</code> (OpenZeppelin)
Overflow/Underflow	Todos	Solidity ^0.8.x (built-in)
Oráculo de preço	ARCAStaking	Chainlink AggregatorV3Interface
Período de lock	ARCAStaking	<code>LOCK_PERIOD = 7 dias</code>

Conclusão

O Protocolo ARCA **não apresenta vulnerabilidades críticas ou de alta severidade**. Os alertas identificados são esperados para contratos desta natureza e estão dentro dos padrões aceitáveis para um MVP educacional. As principais proteções de segurança (`ReentrancyGuard`, `Ownable`, Solidity 0.8.x) estão corretamente implementadas.

Ferramentas utilizadas: Slither (análise estática), Hardhat (compilação e testes)

Repositório: <https://github.com/LRMoraiss/arca-protocol>